PRAKTIKUM CLOUD COMPUTING

DOKUMEN SPESIFIKASI ARSITEKTUR

Sistem Flash Sale E-Commerce

Keterangan	Isi
Mata kuliah	Cloud Computing
Kegiatan	Tugas Terstruktur, Pertemuan 1: Planning & Application Setup
Kelompok	Kelompok 1
Anggota	Afrizal, Ardhiansyah, Luthfion, M. Diwan, Erridho
Kelas	H7D.7
Studi kasus	Sistem Flash Sale E-Commerce
Versi	1.2 (revisi: load balancer, waktu failover, bentrok data setelah failover)
Tanggal	[ISI]

1. Latar Belakang dan Tantangan

Flash sale adalah penjualan singkat dengan stok terbatas, sehingga ribuan pengguna menekan tombol beli hampir bersamaan. Ada lima tantangan teknis yang menjadi dasar rancangan:

1. **Lonjakan traffic ekstrem.** Beban naik tajam dalam waktu singkat.
2. **Overselling.** Beberapa pembeli mendapat barang yang sama padahal stok tinggal sedikit, akibat race condition. Stok dapat menjadi minus.
3. **Pesanan ganda.** Satu niat beli tercatat lebih dari sekali karena klien mengulang request setelah timeout atau crash.
4. **Titik kegagalan tunggal pada database (single point of failure).** Jika satu-satunya server database mati, seluruh layanan ikut mati. Karena itu database dirancang sebagai server primary (master) dan replica (slave).
5. **Titik kegagalan pada pintu masuk.** Jika satu-satunya pintu masuk (load balancer) mati, pengguna tidak dapat mengakses layanan walau backend dan database sehat. Dampak dan pemulihannya dianalisis pada bagian 4.4 dan 8.8.

2. Ruang Lingkup dan Asumsi

- **Termasuk:** daftar produk, checkout, status pesanan, health check, pengecekan integritas data, database primary-replica (master-slave) dengan failover manual, serta analisis kegagalan load balancer.
- **Tidak termasuk:** login, payment gateway sungguhan, keranjang multi-item, pengiriman. Pembayaran disimulasikan, yaitu order langsung berstatus PAID.

- Dua server database PostgreSQL: primary (master) untuk penulisan dan replica (slave) read-only melalui streaming replication asinkron. Failover otomatis (Patroni atau repmgr) dan replikasi lintas lokasi berada di luar lingkup praktikum.
- **Load balancer:** pada Pertemuan 1 (localhost) aplikasi berjalan dengan satu instance backend tanpa load balancer. Load balancer Nginx di depan dua instance backend ditambahkan pada Pertemuan 2 melalui docker-compose. Analisis kegagalannya didokumentasikan sejak sekarang agar rancangan tidak berubah di tengah praktikum.
- user_id dikirim klien tanpa autentikasi. Ini batasan prototipe, bukan praktik produksi.

3. Kebutuhan Sistem

3.1 Kebutuhan Fungsional

ID	Kebutuhan
FR-1	Menampilkan daftar produk, harga, dan sisa stok (dibaca dari replica, fallback ke primary)
FR-2	Pengguna dapat checkout satu produk dengan jumlah tertentu
FR-3	Checkout ditolak jika stok tidak mencukupi
FR-4	Request checkout dengan Idempotency-Key yang sama tidak membuat order kedua
FR-5	Pengguna dapat melihat detail pesanan
FR-6	Health check mencerminkan status database primary (menentukan 200 atau 503) dan melaporkan status replica serta jeda replikasi
FR-7	Endpoint statistik untuk memverifikasi konsistensi stok dan order
FR-8	Semua penulisan dan bacaan yang harus akurat (checkout, detail pesanan, statistik) dilayani oleh primary; replica bersifat read-only
FR-9	Jika primary mati, replica dapat dipromosikan menjadi primary baru (failover manual)
FR-10	Load balancer membagi request ke instance backend yang sehat dan berhenti mengirim ke instance yang gagal (Pertemuan 2)

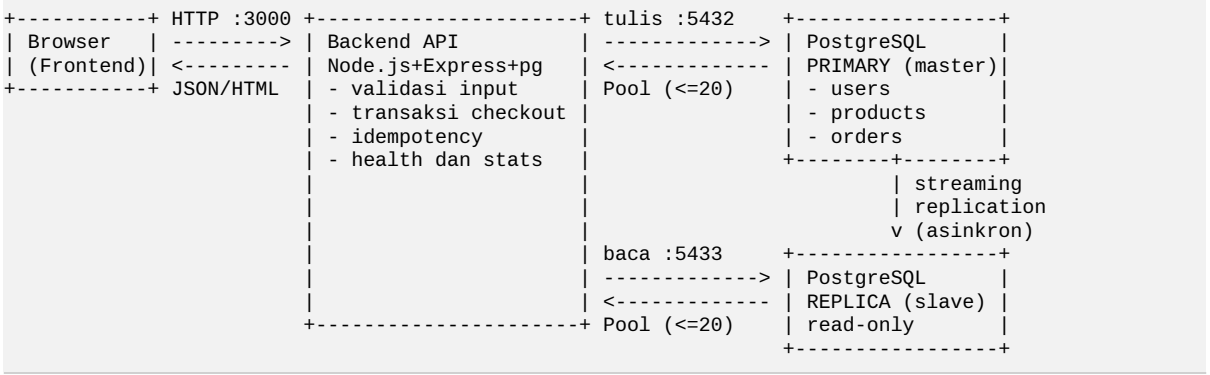
3.2 Kebutuhan Non-Fungsional

ID	Kebutuhan
NFR-1	Stok tidak pernah negatif
NFR-2	initial_stock − stock = total qty order untuk setiap produk
NFR-3	Tidak ada dua order dengan idempotency key yang sama
NFR-4	Backend tidak crash saat database mati; mengembalikan status 503
NFR-5	Target pemulihan: RTO ≤ 5 menit dan RPO ≤ 1 menit (lihat Dokumen DRP)
NFR-6	Dapat diuji sampai 1000 virtual users (Pertemuan 3)
NFR-7	Jika replica mati, /products tetap dilayani dari primary tanpa error ke pengguna
NFR-8	Matinya satu server database tidak mematikan seluruh layanan: saat primary mati, katalog produk tetap terbaca dari replica dan penulisan dipulihkan dengan failover ke replica
NFR-9	Jeda replikasi dilaporkan oleh /health (replica_lag_seconds)
NFR-10	Jika load balancer mati, layanan pulih ≤ 1 menit tanpa kehilangan data (restart policy otomatis; manual sebagai cadangan)

ID	Kebutuhan
NFR-11	Setelah failover, primary lama tidak boleh menerima penulisan lagi (fencing). Data yang hanya ada di primary lama harus dapat diidentifikasi dan direkonsiliasi
NFR-12	Waktu failover primary diukur dan dilaporkan pada DR Test, dengan target ≤ 5 menit

4. Arsitektur Sistem

4.1 Arsitektur Pertemuan 1 (localhost, primary-replica)

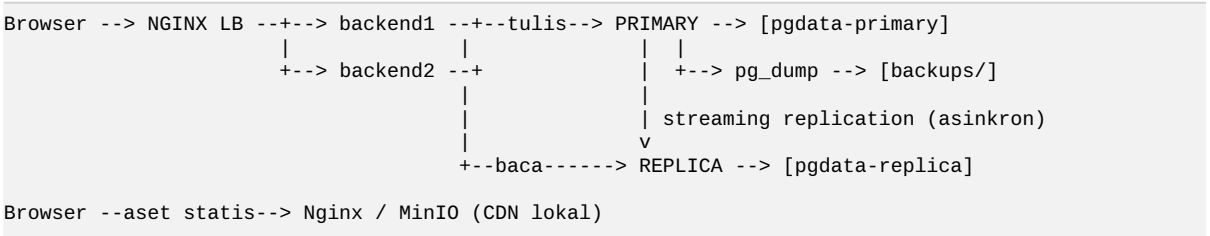


Pada Pertemuan 1, backend juga menyajikan file frontend dari folder frontend. Pada Pertemuan 5, file statis dipindahkan ke Nginx atau MinIO.

Peran kedua server database:

- **Primary (master), port 5432:** menerima semua penulisan dan bacaan yang harus akurat, yaitu /checkout, /orders/:id, dan /stats.
- **Replica (slave), port 5433:** salinan read-only dari primary melalui streaming replication asinkron. /products dibaca dari replica (header respons X-Read-From: replica) sehingga beban baca terpisah dari primary. Stok yang tampil bisa tertinggal sepersekian detik, tetapi keputusan stok tetap diambil di primary saat checkout.
- **Jika replica mati:** pembacaan /products otomatis dialihkan ke primary (X-Read-From: primary). Checkout tidak terpengaruh.
- **Jika primary mati:** /checkout, /orders, dan /stats membalas 503 tanpa menjatuhkan backend, sedangkan /products tetap terbaca dari replica. Replica kemudian dipromosikan menjadi primary baru secara manual (bagian 8.6).

4.2 Target Arsitektur Akhir (setelah Pertemuan 2 sampai 5)



Load balancer Nginx mendengarkan di port 8080. Seluruh layanan berada dalam satu Docker network. Primary dan replica berupa dua container dengan Docker Volume

masing-masing. Backend bersifat stateless (state ada di database), sehingga dua instance backend dapat berjalan di belakang load balancer.

4.3 Pemilihan Teknologi

Komponen	Pilihan	Alasan
Backend	Node.js + Express	Cepat dibuat, I/O non-blocking, mudah dikontainerisasi
Load balancer	Nginx (upstream dua instance backend), restart policy unless-stopped	Ringan, mudah dikonfigurasi, mendukung passive health check (max_fails dan fail_timeout)
Database	PostgreSQL 16	ACID penuh, row-level lock, CHECK dan UNIQUE constraint, pg_dump matang
Replikasi	PostgreSQL streaming replication (asinkron), dibuat dengan pg_basebackup	Memisahkan beban baca dan menyediakan salinan data yang siap dipromosikan saat primary mati
Failover	Manual: pg_ctl promote (scripts/promote-replica.sh)	Failover otomatis memerlukan Patroni atau repmgr, di luar lingkup praktikum
Driver	pg dengan connection pool	Pool membatasi jumlah koneksi ke database (maks 20 per instance backend per server database)
Frontend	HTML + JavaScript vanilla	Fokus praktikum ada di infrastruktur, bukan antarmuka

4.4 Titik Kegagalan per Komponen

Tabel ini merangkum apa yang terjadi jika masing-masing komponen mati, termasuk load balancer. Waktu adalah perkiraan rancangan yang diukur di Pertemuan 4.

Komponen yang mati	Dampak bagi pengguna	Pemulihan	Perkiraan waktu	Data hilang?
Load balancer (Nginx)	Semua request ke port publik gagal; layanan tidak dapat diakses	Otomatis lewat restart policy; manual: docker compose up -d lb	Detik (otomatis); ≤ 1 menit (manual)	Tidak (stateless)
Satu instance backend	Load balancer berhenti mengirim ke instance itu; request yang sedang diproses di sana gagal dan diulang klien	Restart instance	Layanan tetap berjalan	Tidak
Semua instance backend	Load balancer membalas error 502 atau 503	Restart backend	Detik	Tidak
Replica	/products dialihkan ke primary; tidak terlihat oleh pengguna	Bangun ulang replica (setup-replica.sh)	Tidak memengaruhi layanan	Tidak
Primary (master)	Checkout membalas 503; katalog tetap tampil dari replica	Failover manual ke replica	± 3 menit (target ≤ 5 menit)	Sebesar jeda replikasi (kecil)
Primary lama hidup kembali setelah failover	Risiko dua primary dengan data berbeda	Fencing, rekonsiliasi, bangun ulang sebagai replica	Dikerjakan setelah layanan stabil	Tidak jika prosedur 8.7 diikuti

5. Desain Database

```
users (1) ----< orders >---- (1) products
```

Tabel	Kolom	Tipe	Constraint
users	id	SERIAL	PK
users	name	VARCHAR(100)	NOT NULL
products	id	SERIAL	PK
products	name	VARCHAR(150)	NOT NULL
products	price	NUMERIC(12,2)	NOT NULL, ≥ 0
products	initial_stock	INT	NOT NULL, ≥ 0
products	stock	INT	NOT NULL, **CHECK ≥ 0**
orders	id	SERIAL	PK
orders	user_id	INT	NOT NULL, FK ke users
orders	product_id	INT	NOT NULL, FK ke products
orders	qty	INT	NOT NULL, **CHECK > 0**
orders	status	VARCHAR(20)	NOT NULL, default PAID
orders	idempotency_key	VARCHAR(100)	NOT NULL, **UNIQUE**
orders	created_at	TIMESTAMPTZ	NOT NULL, default NOW()

Invarian data (selalu benar jika sistem sehat):

1. stock ≥ 0 untuk semua produk.
2. Tidak ada dua order dengan idempotency_key yang sama.
3. initial_stock − stock = Σ orders.qty per produk. Invarian ini dipakai untuk membuktikan data benar setelah pemulihan.

Skema dibuat di primary dan otomatis tereplikasi ke replica, termasuk constraint CHECK dan UNIQUE. Replica bersifat read-only sehingga tidak dapat menerima penulisan langsung. Ketiga invarian di atas dijaga oleh primary.

6. Spesifikasi API

Method	Endpoint	Fungsi	Respons
GET	/products	Daftar produk dan stok (dari replica, fallback ke primary; header X-Read-From menunjukkan sumber)	200, 503
POST	/checkout	Membuat pesanan. Header Idempotency-Key wajib; body berisi user_id, product_id, qty	201 dibuat 200 duplikat 400 input salah 404 produk tidak ada 409 stok habis 503 DB tidak tersedia
GET	/orders/:id	Detail pesanan (dari primary)	200, 400, 404, 503

Method	Endpoint	Fungsi	Respons
GET	/health	Status server, database primary (menentukan 200 atau 503), dan replica (status serta replica_lag_seconds)	200, 503
GET	/stats	Cek konsistensi stok terhadap order (dari primary)	200, 503

Penulisan (/checkout) dan bacaan yang harus akurat (/orders/:id, /stats) selalu ke primary. Hanya /products yang dibaca dari replica. Respons 503 pada /products hanya terjadi jika replica dan primary sama-sama tidak tersedia. Jika dipasang load balancer, request melewati load balancer terlebih dahulu dan kontrak API di atas tidak berubah.

7. Alur Checkout dan Alur Baca

7.1 Alur Checkout (penulisan, ke primary)

1. Klien mengirim POST /checkout dengan Idempotency-Key (UUID unik per niat beli).
2. Backend memeriksa di primary apakah key sudah pernah dipakai. Jika sudah, kembalikan order lama (200 DUPLICATE).
3. Backend membuka transaksi dan mengurangi stok secara atomik dengan query di bawah.

```
UPDATE products SET stock = stock - qty
 WHERE id = ? AND stock >= qty;
```

1. Jika tidak ada baris terpengaruh: ROLLBACK, kembalikan 409 SOLD_OUT (atau 404 bila produk tidak ada).
2. Jika berhasil: INSERT order, COMMIT, kembalikan 201 CREATED.
3. Jika database tidak tersedia: kembalikan 503. Klien boleh mengulang dengan key yang sama dengan aman.

7.2 Alur Baca Daftar Produk (ke replica)

Backend mengirim SELECT ke replica. Jika berhasil, respons dikirim dengan header X-Read-From: replica. Jika replica gagal (mati atau tidak terjangkau), backend mencatat peringatan lalu mengulang SELECT yang sama ke primary (X-Read-From: primary). Jika primary dan replica sama-sama tidak tersedia, kembalikan 503.

Karena replikasi asinkron, stok yang tampil bisa tertinggal sepersekian detik dari primary. Hal ini tidak menyebabkan overselling karena keputusan stok selalu diambil oleh UPDATE atomik di primary saat checkout (bagian 8.1).

8. Strategi Pencegahan Masalah Inti

8.1 Overselling

Pendekatan naif, yaitu membaca stok lalu memperbaruinya, rentan race condition. Dua request sama-sama membaca stok 1, lalu keduanya mengurangi sehingga stok menjadi −1.

Pendekatan naif (salah)	Pendekatan yang dipakai (benar)
A: SELECT stock → 1 B: SELECT stock → 1 A: UPDATE stock = 0 B: UPDATE stock = −1 (overselling)	A: UPDATE ... WHERE stock >= 1 → mengunci baris, stok 1 menjadi 0 B: UPDATE ... menunggu kunci baris dari A A: COMMIT, kunci dilepas B: kondisi WHERE dievaluasi ulang pada stok 0 → 0 baris terpengaruh → ditolak (409)

Pada level isolasi default READ COMMITTED, PostgreSQL mengevaluasi ulang kondisi WHERE terhadap versi baris terbaru setelah menunggu kunci. Karena itu hasilnya benar tanpa SELECT ... FOR UPDATE terpisah.

8.2 Pertahanan Berlapis

Lapisan	Mekanisme	Fungsi
Query	WHERE stock >= qty	Mencegah stok minus dalam kondisi normal
Skema	CHECK (stock >= 0)	Database menolak stok minus jika ada bug di kode
Transaksi	Update stok dan insert order dalam satu transaksi	Tidak ada kondisi stok berkurang tanpa order tercatat
Skema	UNIQUE (idempotency_key)	Database menolak order kedua dengan key yang sama
Replikasi	Replica streaming dari primary	Salinan data siap dipromosikan jika primary mati, sekaligus melayani bacaan
Read failover	readQuery() beralih ke primary jika replica gagal	Bacaan produk tetap berjalan saat replica mati
Fencing	Primary lama dihentikan dan tidak boleh restart otomatis setelah failover	Mencegah dua primary menerima penulisan (bagian 8.7)
Audit	Endpoint /stats dan query verifikasi	Mendeteksi ketidakkonsistenan setelah insiden

8.3 Pesanan Ganda

Jika dua request dengan key yang sama masuk bersamaan, request kedua mendapat error unique violation (kode 23505) saat INSERT. Kode membatalkan transaksinya, sehingga pengurangan stok ikut batal, lalu mengembalikan order milik request pertama. Hasilnya satu order dan stok berkurang satu kali.

8.4 Saat Database Crash

Saat crash, klien tidak tahu apakah pesanannya sudah tersimpan. Karena key-nya sama saat mencoba lagi, hasilnya aman dalam dua kemungkinan: jika order sudah tersimpan, server menjawab 200 DUPLICATE; jika belum, server membuatnya (201). Inilah alasan idempotency key dirancang sejak Pertemuan 1. Jika yang mati adalah server primary, penanganannya dijelaskan pada bagian 8.6.

8.5 Replikasi Primary-Replica (Master-Slave)

Replica dibuat dengan pg_basebackup (scripts/setup-replica.sh) dan menerima perubahan dari primary melalui streaming replication. Mode replikasi asinkron: primary tidak menunggu konfirmasi replica sebelum COMMIT, sehingga latensi checkout tidak bertambah.

Konsekuensinya ada jeda replikasi (replication lag) yang dilaporkan /health pada field replica_lag_seconds. Karena checkout selalu ke primary, jeda ini hanya memengaruhi stok yang tampil di daftar produk, bukan keputusan stok.

Replica bukan pengganti backup: perintah destruktif seperti DROP TABLE ikut tereplikasi ke replica. Karena itu pg_dump tetap dipakai (lihat Dokumen DRP).

8.6 Saat Primary (Master) Mati: Failover Manual dan Lama Prosesnya

Dengan primary-replica, matinya satu server database tidak mematikan seluruh layanan. Urutan penanganan dan perkiraan waktunya:

Tahap	Aksi	Perkiraan
1. Deteksi	/health membalas 503 dua kali berturut-turut (skrip pemantau mengecek tiap detik); operator menyatakan insiden dan mencatat T0	0,5 menit
2. Diagnosis dan fencing	Cek /health (primary down, replica up, jeda replikasi kecil). Pastikan container primary lama berhenti dan tidak akan restart otomatis	0,75 menit
3. Promosi replica	scripts/promote-replica.sh (pg_ctl promote). Proses promosinya sendiri hanya beberapa detik; replica membuka timeline baru dan dapat ditulisi	0,25 menit
4. Pindah koneksi backend	Ubah .env backend (DB_PORT ke port replica, DB_REPLICA_HOST dikosongkan), lalu jalankan ulang backend (semua instance)	0,5 menit
5. Validasi	Jalankan query verifikasi dan /stats pada primary baru	1 menit
Total	Dihitung dari T0 sampai /health kembali ok (T1)	**± 3 menit**

- **Dampak selama proses:** /checkout, /orders, dan /stats membalas 503 selama kira-kira lama proses di atas. /products tetap terbaca dari replica. Klien mengulang request yang gagal dengan Idempotency-Key yang sama setelah layanan pulih.
- **Data yang berisiko:** order yang sudah commit di primary tetapi belum sampai ke replica (replikasi asinkron). Besarnya sama dengan jeda replikasi saat primary mati, yang kecil pada kondisi normal.
- **Asumsi:** operator siaga selama flash sale dan prosedur dijalankan lewat skrip. Jika operator tidak siaga, waktu bertambah sebesar keterlambatan deteksi.
- **Pembanding:** failover otomatis (Patroni atau repmgr) umumnya selesai dalam puluhan detik, tetapi memerlukan komponen tambahan dan berada di luar lingkup praktikum.
- **Status angka:** ± 3 menit adalah perkiraan rancangan. Nilai sebenarnya diukur pada Uji C di Pertemuan 4 dan dilaporkan di Dokumen DRP.

Failover dibuat manual karena failover otomatis memerlukan Patroni atau repmgr yang berada di luar lingkup praktikum. Karena replikasi asinkron, order yang baru commit di primary dan belum sampai ke replica saat primary mati dapat hilang (RPO lebih dari 0,

kecil). Mitigasi di produksi adalah replikasi sinkron dengan konsekuensi latensi checkout bertambah.

8.7 Primary Lama Hidup Kembali: Risiko Bentrok Data (Split-Brain)

Pertanyaan: jika master mati, lalu hidup lagi, apakah ada bentrok data karena replica sudah menjadi master?

Jawaban: ada risiko, dan risikonya bergantung pada kapan master lama hidup kembali.

Kondisi	Akibat	Penanganan
Master lama hidup kembali **sebelum** replica dipromosikan (misalnya hanya restart singkat)	Tidak ada bentrok. Replica tersambung lagi ke primary dan melanjutkan streaming. Failover tidak perlu dijalankan	Cek /health sampai replica up dan jeda replikasi kembali kecil
Master lama hidup kembali **setelah** replica dipromosikan	Ada dua server yang merasa primary dan data keduanya berbeda (split-brain). Jika backend atau pengguna menulis ke master lama, perbedaan makin besar	Fencing dan rekonsiliasi (di bawah)

Mengapa data bisa berbeda

- Setelah promosi, replica membuka timeline baru dan menerima penulisan. Master lama masih berada di timeline lama dan menganggap dirinya primary.
- **Order ekor:** order yang sudah commit di master lama tetapi belum sampai ke replica sebelum master mati. Pembeli sempat menerima 201, namun order itu tidak ada di primary baru.
- **Order baru:** order yang masuk ke primary baru selama failover tidak ada di master lama.
- PostgreSQL tidak punya replikasi multi-master bawaan, sehingga dua versi data ini tidak dapat digabung otomatis. Master lama tidak boleh sekadar disambungkan kembali sebagai replica tanpa dibangun ulang.

Apa yang tetap aman

- Primary baru tetap konsisten: replica menerapkan perubahan berurutan dan setiap transaksi atomik, sehingga stok dan order di sana saling cocok. Tidak pernah ada stok minus karena CHECK (stock >= 0).
- Yang berisiko hanya order ekor. Karena stoknya belum terpotong di primary baru, stok itu bisa terjual ke pembeli lain sebelum order ekor diputar ulang.

Pencegahan (fencing)

Lapis	Tindakan
1	Saat failover, hentikan container primary lama dan nonaktifkan restart otomatisnya (docker stop dan docker update --restart=no)
2	Backend hanya diarahkan ke satu alamat tulis, yaitu primary baru (.env). Master lama tidak pernah menerima penulisan dari aplikasi
3	Master lama yang hidup kembali dijalankan terisolasi (port lain, tidak dikenal backend) dan dipakai hanya untuk membaca data yang tertinggal

Penanganan jika master lama sudah hidup kembali

1. Jalankan master lama secara terisolasi dan jangan arahkan backend ke sana.
2. Bandingkan timeline dan isi data: timeline_id berbeda menandakan divergensi, dan daftar idempotency_key yang hanya ada di master lama menunjukkan order ekor.
3. Putar ulang order ekor ke primary baru melalui POST /checkout dengan Idempotency-Key yang sama. Hasil 201 berarti order diselamatkan, 200 berarti ternyata sudah ada, dan 409 berarti stok sudah habis sehingga pesanan perlu dikompensasi (refund atau pemberitahuan). Karena idempotency key UNIQUE, tidak ada order ganda.
4. Setelah rekonsiliasi dicatat, hapus data master lama dan bangun ulang sebagai replica dari primary baru (scripts/setup-replica.sh dengan pg_basebackup).
5. Jalankan validasi integritas pada primary baru.

Langkah dan perintah rinci ada di Dokumen DRP bagian 7.4. Replikasi sinkron dapat menghilangkan order ekor (RPO = 0) tetapi menambah latensi checkout dan menghentikan penulisan jika replica mati, sehingga tidak dipilih untuk praktikum ini.

8.8 Load Balancer Mati

Pertanyaan: bagaimana jika load balancer mati?

Aspek	Penjelasan
Peran	Nginx menerima semua request dari pengguna dan membagi ke dua instance backend. Load balancer tidak menyimpan state atau data
Dampak jika mati	Pengguna tidak dapat mengakses layanan sama sekali walau backend dan database sehat. Pada rancangan satu load balancer, ini adalah titik kegagalan tunggal pada lapisan masuk
Data	Tidak ada data hilang (RPO = 0) karena load balancer tidak menyimpan data. Request yang sedang berjalan saat load balancer mati gagal
Request yang gagal	Klien mengulang request dengan Idempotency-Key yang sama setelah layanan pulih. Aman karena idempotent. Nginx tidak mengulang otomatis request POST yang sudah terkirim ke backend, sehingga pengulangan dilakukan klien
Pemulihan otomatis	Container load balancer memakai restart policy unless-stopped. Jika prosesnya crash, Docker menjalankannya kembali dalam hitungan detik
Pemulihan manual	Jika container terhapus atau dihentikan manual (restart policy tidak berlaku): docker compose up -d lb. Perkiraan ≤ 1 menit termasuk waktu operator
Jalur darurat	Selama load balancer pulih, backend dapat diakses langsung lewat portnya sebagai sementara (manual). Cukup untuk demonstrasi, bukan solusi produksi
Satu backend mati	Dengan max_fails dan fail_timeout, Nginx menandai instance gagal sebagai tidak sehat dan mengirim trafik ke instance lain. Layanan tidak berhenti
Produksi	Dua node load balancer dengan virtual IP (keepalived atau VRRP), atau load balancer terkelola milik penyedia cloud. Di luar lingkup praktikum
Pengukuran	Waktu pemulihan diukur pada Uji D di Pertemuan 4: hentikan container load balancer saat beban berjalan, catat waktu sampai request kembali berhasil

9. Keamanan Dasar

Aspek	Penanganan di prototipe	Keterbatasan
SQL injection	Semua query berparameter ($1, $2)	Aman untuk input yang dikirim
Validasi input	user_id, product_id, qty harus integer; qty ≥ 1; key wajib	Belum ada batas atas qty
Autentikasi	Tidak ada	Produksi wajib memakai login atau token
Rate limiting	Tidak ada	Produksi perlu pembatasan request per IP atau pengguna (dapat dipasang di load balancer)
Rahasia	Kredensial DB di file .env yang masuk .gitignore	File .env tidak boleh di-commit
Akses replikasi	pg_basebackup dan replikasi memakai user postgres di localhost	Produksi memakai role REPLICATION khusus, pg_hba.conf terbatas, dan TLS

10. Risiko Arsitektur

Risiko	Dampak	Mitigasi
Server primary mati (sebelumnya: database tunggal sebagai single point of failure)	Penulisan (checkout) berhenti; katalog produk tetap terbaca dari replica	Replica siap dipromosikan (failover manual), pg_dump, dan prosedur di Dokumen DRP
Load balancer tunggal mati	Seluruh layanan tidak dapat diakses; data tidak hilang	Restart policy, prosedur manual, klien retry idempotent; redundansi dua node di luar lingkup
Primary lama hidup kembali setelah failover (split-brain)	Dua primary dengan data berbeda; order ekor hilang atau bentrok	Fencing, rekonsiliasi lewat idempotency key, bangun ulang sebagai replica (bagian 8.7)
Failover masih manual	Waktu pemulihan bergantung pada operator	Skrip promote-replica.sh dan prosedur tertulis di DRP; waktunya diukur di DR Test (Pertemuan 4)
Jeda replikasi (asinkron)	Stok di daftar produk tertinggal; order terakhir bisa hilang saat failover	Checkout selalu ke primary; /health melaporkan lag; retry idempotent; replikasi sinkron di luar lingkup
Primary dan replica di satu host	Kegagalan host mematikan keduanya	Di luar lingkup praktikum; produksi menempatkan replica di host atau zona berbeda
Hot row pada produk populer	Latensi tinggi saat beban puncak	Dianalisis di Pertemuan 3
Pool koneksi terbatas	Timeout saat 1000 VU	Dituning di Pertemuan 3
Tanpa autentikasi	Penyalahgunaan user_id	Di luar lingkup, dicatat sebagai batasan

11. Rencana Pengembangan

Pertemuan	Pengembangan
1	Arsitektur, DRP awal, aplikasi di localhost dengan database primary-replica; analisis kegagalan load balancer, failover, dan split-brain
2	Dockerfile, docker-compose (Nginx load balancer, dua instance backend, primary, replica), Docker Volume untuk tiap database
3	Stress test 10, 100, 500, 1000 VU dan analisis bottleneck (termasuk dampak baca dari replica dan beban lewat load balancer)
4	Backup otomatis, DR Test: failover ke replica (Uji C), load balancer mati (Uji D), primary lama hidup kembali (Uji E), recovery
5	Pemisahan aset statis ke Nginx atau MinIO (CDN)

Riwayat Revisi

Versi	Perubahan
1.0	Versi awal (Pertemuan 1)
1.1	Revisi pertama: arsitektur primary-replica (master-slave), failover manual
1.2	Revisi kedua: load balancer dan kegagalannya (4.4, 8.8), lama proses failover (8.6), bentrok data setelah failover (8.7), tanggapan atas masukan dosen